Blending Serverful and Serverless Cloud Resources for Cost-Effective Workflow Execution

Quanwang Wu, *Senior Member, IEEE*, Qixin Zhou, Ruyi Sun, Mengchu Zhou, *Fellow, IEEE*, Ji Feng, Taolin Guo, and Chao Chen, *Senior Member, IEEE*

***Abstract*—Serverless computing offers fine-grained billing and elastic scalability, making it appealing for workflow execution. However, it also suffers from cold-start latency and stricter execution constraints. In contrast, traditional serverful cloud resources, such as virtual machines, provide coarser provisioning granularity but benefit from relatively lower unit cost. This work explores the potential of blending serverful and serverless resources to harness their complementary strengths for cost-effective workflow execution. We propose a hybrid resource management framework that dynamically allocates workflow tasks across both types of resources. A Budget-constrained Workflow scheduling algorithm for Blended cloud (BWB) is developed to minimize makespan while respecting user-specified budget. Evaluation experiments are conducted under real-world cloud settings by using realistic workflow applications. BWB is compared against state-of-the-art approaches for serverful, serverless, and blended clouds. Experimental results show that BWB consistently outperforms its state-of-the-art peers, achieving makespan reductions ranging from 10.6% to 37.6%, thereby demonstrating the cost-effectiveness of blending cloud resources for workflow execution.**

***Note to Practitioners*—Workflow scheduling in cloud environments is of significant practical relevance and has drawn considerable interest from both industry and academia. Blending serverful and serverless cloud resources offers a promising path toward cost-effective workflow execution, yet it introduces additional challenges in resource coordination, performance variability, and budget control. This work presents a hybrid resource management framework together with a budget-constrained workflow scheduling algorithm designed specifically for blended cloud environments. The proposed approach has been validated through real-world experiments and further assessed via comprehensive simulation studies. Results demonstrate that it consistently outperforms state-of-the-art methods in terms of makespan under user-defined budget limits. The proposed solution is practical and readily applicable in real cloud settings. It can assist cloud users and operators in efficiently allocating workflow tasks across heterogeneous cloud resources to achieve better performance within a specified budget.**

***Index Terms*—Cloud computing, constrained optimization, serverless computing, workflow scheduling.**

I. Introduction

In RECENT years, serverless computing has rapidly gained popularity as a cloud-native execution model due to its ease of use, scalability, and fine-grained billing [1]. By allowing developers to deploy serverless functions (SFs) without managing servers, serverless platforms have been used to greatly simplify the development and operational process, making them ideal for event-driven and highly dynamic workloads. However, it also faces several limitations. Most notably, it suffers from cold start latency, which occurs when function containers are initialized before execution begins [2]. In addition, SFs are subject to strict execution constraints. For instance, AWS Lambda,[1] one of the most widely used serverless platforms, imposes a maximum execution time of 15 minutes, disk space limited to 10 GB, and memory allocation of up to 10 GB.

In contrast, traditional serverful cloud resources—such as virtual machines (VMs)—offer users full control over the execution environment and can provide lower unit cost when fully utilized [3], [4]. However, they also exhibit coarser provisioning granularity, often imposing a minimum billing time (MBT) of at least one minute. As a result, they may suffer from resource underutilization and higher operational overhead, particularly under dynamic or bursty workloads.

Workflow applications consist of multiple interdependent tasks organized according to a directed acyclic graph, where each node represents a computational task and edges denote data or control dependencies [5]. These applications are widely used in domains such as scientific computing, bioinformatics, data analytics, and machine learning pipelines. These days, more and more workflow applications have been deployed in cloud platforms due to their scalability, flexibility, and pay-as-you-go pricing models.

Due to the complex structure and varying parallelism degree in different phases of a workflow, can we blend serverless and serverful cloud resources for workflow execution to significantly reduce both cost and execution time (i.e., makespan)? Let us answer it via an illustrative example as shown in Fig. 1. Fig. 1(a) presents a workflow instance including 4 tasks. When these tasks are deployed in serverless and serverful

[1]https://aws.amazon.com/lambda/

Received 2 December 2025; revised 7 March 2026; accepted 4 May 2026. Date of publication 11 May 2026; date of current version 28 May 2026. This article was recommended for publication by Associate Editor M. T. Khan and Editor C.-Y. Lee upon evaluation of the reviewers' comments. This work was supported in part by the National Natural Science Foundation of China under Grant 62322601, Grant U22A2026, and Grant 62172065; in part by the National Science and Technology Major Project under Grant 2025ZD1700805; and in part by the Fundo para o Desenvolvimento das Ciencias e da Tecnologia (FDCT) under Grant 0095/2025/AMJ. *(Corresponding author: Chao Chen.)*

Quanwang Wu, Qixin Zhou, Ruyi Sun, and Chao Chen are with the College of Computer Science, Chongqing University, Chongqing 401331, China (e-mail: wqw@cqu.edu.cn; cqu_zqx@stu.cqu.edu.cn; ruyisun@stu.cqu.edu.cn; cschaochen@cqu.edu.cn).

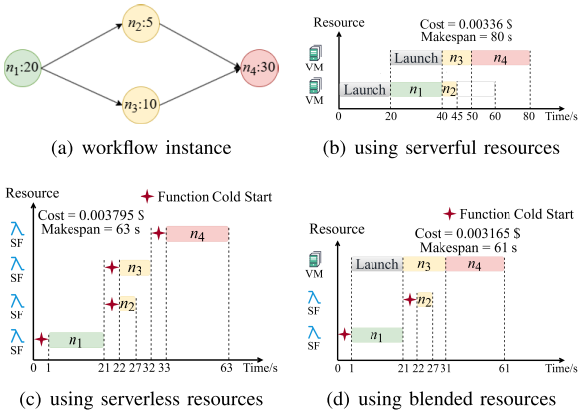
Mengchu Zhou is with the Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ 07102 USA, and also with Macao Institute of Systems Engineering and Collaborative Laboratory for Intelligent Science and Systems, Macau University of Science and Technology, Macao 999078, China (e-mail: mengchu.zhou@njit.edu).

Ji Feng and Taolin Guo are with the College of Computer Science, Chongqing Normal University, Chongqing 401331, China (e-mail: jifeng@cqnu.edu.cn; tguo@cqnu.edu.cn).

Digital Object Identifier 10.1109/TASE.2026.3692157

1558-3783 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:20:00 UTC from IEEE Xplore. Restrictions apply.



(a) workflow instance (b) using serverful resources

(c) using serverless resources (d) using blended resources

Fig. 1. Workflow scheduling with different cloud resources.

cloud resources with identical hardware configuration, their execution time is 20s, 5s, 10s, and 30s, respectively, with inter-task transmission time being neglected. The corresponding execution cost with SFs is 0.0011$, 0.000275$, 0.00055$, and 0.00165$, respectively. The VM's unit cost is 0.000028$/s with an MBT of 60s.[2] The cold start latency for each SF is 1s and incurs 0.000055$, whereas VMs require a billable launch-up time of 20 seconds. Figs. 1(b)-(d) illustrate the scheduling solutions by using serverful, serverless and blended cloud resources, respectively. The results demonstrate that blending serverless and serverful resources yields the best solution in terms of both cost and makespan, reducing them by up to 16.60% and 23.75%, respectively.

Workflow scheduling is a classic NP-Hard problem in distributed computing [6]. Scheduling workflows in a blended cloud environment becomes even more challenging due to the fundamentally distinct billing models of serverful and serverless resources, as well as the complexity involved in dynamically provisioning both types of resources for workflow execution. Up to now, substantial research efforts have been devoted to the cloud workflow scheduling problems. However, most existing approaches are designed for the serverful or serverless cloud environments but not blended one [7]. Only a few studies [8], [9] have coped with the latter despite its growing practical significance. However, [8] assumes a fixed number of VMs and static configurations for SFs; while [9] does not account for the startup latency of VMs and SFs. Hence, their approaches are of only limited practical applicability. There is an urgent need for an effective workflow scheduling approach tailored to the characteristics of blended cloud environments and various user requirements.

This paper presents a workflow scheduling framework that integrates serverful and serverless cloud resources. We propose a Budget-constrained Workflow scheduling algorithm for Blended cloud (BWB), which aims to minimize the makespan under a user-specified budget by leveraging the complementary advantages of both resource types. Specifically, BWB first constructs an initial solution using VM resources, then reduces the overall cost by strategically replacing selected VMs with SF resources, and finally iteratively refines the configurations of critical-path tasks to enhance performance given a budget constraint. Evaluation experiments are conducted with real-world cloud settings by using representative workflow applications. BWB is compared against four state-of-the-art approaches targeting serverful, serverless, and blended cloud environments. Experimental results show that BWB achieves significant makespan reductions compared to its counterparts.

The remainder of this paper is organized as follows. Section II reviews the related work. Section III presents the problem formulation and scheduling model. Section IV describes the proposed scheduling algorithm in detail. Section V reports the experimental evaluation and results. Finally, Section VI concludes this paper.

[2]https://aws.amazon.com/ec2/pricing/on-demand/

II. Related Work

Workflow scheduling in serverful cloud platforms was a major research focus over the past decade. In recent years, with the widespread adoption of serverless computing, an increasing number of workflows have been deployed on serverless platforms. Accordingly, we review the related work from three perspectives: 1) workflow scheduling in serverful cloud environments, 2) workflow scheduling in serverless cloud environments, and 3) workflow scheduling in blended cloud environments.

A. Workflow Scheduling in Serverful Cloud

Minimizing workflow makespan and resource cost is critical when scheduling workflows on serverful cloud platforms. Due to the NP-hardness of the workflow scheduling problem, most existing approaches rely on heuristic strategies, and providing provable theoretical guarantees is generally non-trivial, even though related theoretical analyses have been studied in other domains (e.g., [10]). Specifically, existing research in this area can be broadly categorized into two groups: constrained optimization [11], [12], [13], [14], [15] and multi-objective optimization [16], [17], [18].

The former primarily encompasses budget-constrained makespan optimization and deadline-constrained cost optimization. For instance, Faragardi et al. [11] rank resources by their efficiency rates by using a greedy selection strategy and introduce the Greedy Resource Provisioning and Heterogeneous Earliest Finish Time (GRP-HEFT) method to minimize makespan under a given budget. Similarly, Fan et al. [12] propose the PACP-HEFT (Priority Adjustment and Critical Path-based HEFT) method for budget-constrained makespan optimization. Sun et al. [13] propose a multi-tree genetic programming hyper-heuristic to optimize workflow makespan across multiple cloud platforms while respecting deadline constraints. Shu et al. [14] introduce a workflow scheduling method that is aware of communication contention in geo-distributed data centers, aiming to minimize makespan while satisfying data privacy constraints.

For the latter, multiple objectives are optimized. For example, Rizvi et al. [16] propose a modified fuzzy adaptive genetic algorithm to minimize both makespan and cost for workflows deployed on VMs. Qin et al. [17] introduce a memetic algorithm to jointly optimize the makespan, cost, and reliability for workflow execution in a multi-cloud environment.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:20:00 UTC from IEEE Xplore. Restrictions apply.

B. Workflow Scheduling in Serverless Cloud

In serverless computing, SFs are independently operated, event-driven, and infrastructure-agnostic. The underlying resource management ecosystem is generally opaque to users. Predicting the performance and cost of serverless workflows is generally more complex than for traditional workflows. Several approaches have been proposed to analyze performance and cost characteristics of serverless workflows prior to optimization [19], [20], [21], [22]. For example, Lin and Khazaei [19] develop analytical models to estimate the average end-to-end response time and cost of serverless workflows. Based on these models, a heuristic algorithm is proposed to configure memory settings for optimizing response time and cost. Similarly, Li et al. [20] propose an Urgency-based Workflow Configuration (UWC) heuristic and a particle swarm optimization method to optimize the trade-off between makespan and cost. Jia et al. [21] introduce FaaSPR, a serverless scheduling system that leverages placement and routing optimizations to minimize workflow processing latency.

Some studies treat serverless workflow scheduling as an optimization problem with inaccessible internal structure, where optimizing scheduling decisions are made indirectly by using heuristic algorithms or machine learning techniques based solely on input–output feedback data [23]. Zhou et al. [24] present Aquatope, a QoS- and uncertainty-aware resource scheduler for serverless workflows, in which workflow cost and makespan values can only be observed by sampling different resource configurations. A customized Bayesian optimization approach is employed to guide a configuration process. Similarly, Jin et al. [25] propose an automated framework that decouples CPU and memory resources, enabling more flexible and efficient provisioning for serverless workflows using an extended Bayesian optimization method.

C. Workflow Scheduling in Blended Cloud

Owing to the complementary characteristics of serverless and serverful cloud resources, researchers have explored their integration to deliver improved cloud services [26]. Raza et al. [27] propose a balanced approach called LIBRA to leverage both VM-based and serverless resources. By monitoring application demands, LIBRA dynamically provisions appropriate VM and serverless instances to minimize operational cost while satisfying service level agreement constraints. Peri et al. [28] propose a hybrid cloud scheduler for executing batch-processing applications in heterogeneous environments, such as private edge infrastructure and public serverless cloud platforms. Mampage et al. [29] propose a hybrid scheduling approach that leverages the benefits of both serverless and serverful clouds by using an actor-critic-based deep reinforcement learning architecture. Experimental results show that their approach reduces user cost by up to 44% and improves application performance by up to 11%.

As mentioned above, there are limited studies investigating workflow scheduling in blended cloud environments. For example, Roy et al. [8] propose an approach called Mashup, which leverages both VM-based and serverless resources to execute scientific workflows. Experimental results show that it achieves average reductions of 34% in execution time and 43% in cost. However, a key limitation of this study is that neither the number of VMs nor configurations of serverless functions within the blended cloud platform can be adjusted dynamically. Zhang et al. [9] propose the Budget Constrained Workflow Scheduling (BCWS) algorithm to minimize makespan under a limited budget in blended cloud environments. However, they overlook several critical challenges, including the startup latency of VMs and SFs as well as the complexity of their respective billing mechanisms. In contrast to these prior studies, this work fully captures the operational characteristics of blended cloud platforms for workflow execution, and proposes an efficient heuristic algorithm to address the budget-constrained optimization problem.

TABLE I
KEY NOTATIONS

Category	Notation	Definition
Resource	V	set of virtual machines
	v_k	the k-th virtual machine
	θ_k	the type of v_k
	w	bandwidth between cloud resources
Workflow	N, E	sets of tasks and edges in a workflow
	n_i	the i-th task, $n_i \in N$
	$e_{i,j}$	precedence dependence from n_i to n_j
	$*n_i$, n_i*	n_i's immediate predecessors and successors
	n_{entry}, n_{exit}	dummy entry and exit tasks
Scheduling	$r_{i,\theta}$	runtime of n_i in VM with type θ
	$\tilde{r}_{i,m}$	runtime of n_i in SF with m GB memory
	$t_{\text{o}}(n_i)$	n_i's start time
	$t_{\bullet}(n_i)$	n_i's finish time
	Z	scheduling solution
	B	budget of executing a workflow

III. SCHEDULING MODEL IN BLENDED CLOUD

In this section, we first introduce workflow applications and blended cloud environment, then describe a workflow scheduling model and elaborate on the scheduling problem under investigation. Key notations are listed in Table I for ease of reference.

A. Workflow Application and Blended Cloud

A workflow application can be modeled as a directed acyclic graph (DAG), denoted as DAG = $\langle N, E\rangle$, where N is the set of nodes corresponding to individual tasks, and E is the set of directed edges indicating precedence dependencies among tasks. Each edge $e_{i,j} \in E$ is associated with a data volume $q_{i,j}$. An edge from task n_i to task n_j means that n_j is allowed to begin only after n_i has completed execution and the associated data has been successfully transferred. Additionally, the notations $*n_i$ and n_i* are used to denote the immediate predecessors and successors of n_i, respectively. To ensure that the workflow has a single entry and a single exit point, two dummy tasks, n_{entry} and n_{exit}, are introduced at the beginning and end of the workflow.

For serverful cloud resources, launching a new VM instance incurs a startup latency of 0.3 to 2 minutes, which is substantially longer than the 0.1 to 5 seconds typically required to start an SF. Executing a task on an existing VM instance requires the instance to be idle and not occupied by other

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:20:00 UTC from IEEE Xplore. Restrictions apply.

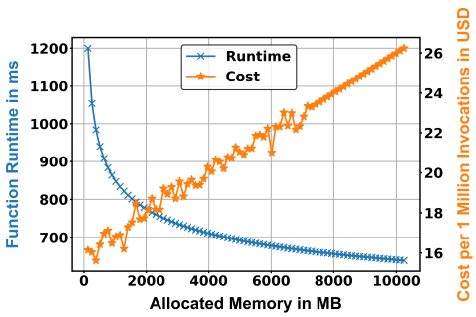


Fig. 2. Runtime and cost of an SF with varying memory sizes.

tasks. Let $\mathcal{T}$ denote the set of available VM types, and $V = \{v_1, v_2, \ldots, v_{|V|}\}$ denote the set of VM instances used for executing the workflow. The type of the k-th VM instance v_k is denoted as θ_k, and the unit price of v_k is determined by its type, denoted as $\eta(\theta_k)$. The required computation time of task n_i on v_k is denoted as r_{i,θ_k}. The cost of v_k depends on its total usage time and unit price. If the usage time is less than MBT, the cost is calculated based on MBT. Therefore, the monetary cost of using VM instance v_k is:

$$c_k = \max\{\text{MBT}, \lceil t_\bullet(v_k) - t_\text{o}(v_k)\rceil\} \times \eta(\theta_k), \tag{1}$$

where $t_\bullet(v_k)$ and $t_\text{o}(v_k)$ denote the lease end time and lease start time of v_k, respectively. Specifically, $t_\bullet(v_k)$ corresponds to the finish time of the last task deployed on v_k, while $t_\text{o}(v_k)$ is obtained as the start time of the first task deployed on v_k minus the VM's launch latency l_VM.

When using serverless cloud resources, a task is executed in the form of an SF. In today's mainstream serverless platforms, memory configuration is the primary parameter for tuning the performance and cost of SFs, while other physical resources such as CPU power, disk performance and network throughput are provisioned proportionally to the allocated memory. The memory size for SFs can typically be configured within a finite range. For example, in AWS Lambda, it can be allocated between 128MB and 10,240MB in 64MB increments. Suppose that the performance of each task has been profiled in advance by using analytical modeling, empirical modeling or historical data [19], [20]. Let $\tilde{r}_{i,m}$ denote the runtime of task n_i on an SF with m-GB memory.

SFs suffer from cold start latency, which occurs when SF containers are initialized before execution begins. Let $\tilde{l}_{i,m}$ be the latency of n_i in an SF with m GB memory. The invocation cost of SFs is generally calculated based on their execution time and cold start latency, i.e.,

$$\tilde{c}_{i,m} = (\tilde{l}_{i,m} + \tilde{r}_{i,m}) \times m \times PGS + PPI, \tag{2}$$

where, PGS refers to the cost per second per GB of memory, and PPI denotes the cost per invocation of a task.

The runtime of SFs decreases as the allocated memory increases. Fig. 2 depicts the runtime and cost of a CPU-intensive SF (hashing) under different memory configurations ranging from 128MB to 10240MB. The runtime–memory curve exhibits a smooth downward trend, while the cost–memory curve rises with irregular fluctuations.

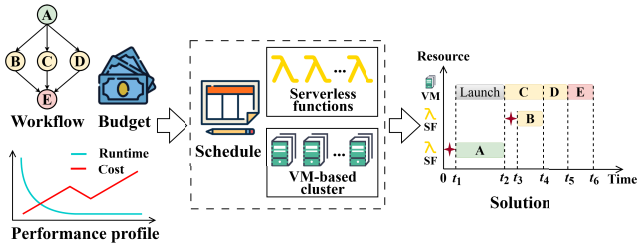


Fig. 3. The workflow scheduling framework in a blended cloud.

Due to the strict execution constraints of serverless platforms, only memory configurations that satisfy these constraints are considered in the following discussions. For example, if the runtime of a given task under a specific memory configuration exceeds the maximum allowed execution time of the serverless platform, that memory configuration is excluded for the task.

B. Scheduling Model and Problem Formulation

The workflow scheduling framework in a blended cloud environment is illustrated in Fig. 3. Each task in a workflow is assigned to either a serverless or serverful cloud platform. For task n_i, we use a boolean variable p_i to indicate the selected platform: $p_i = 1$ if n_i is assigned to an SF, and $p_i = 0$ if it is assigned to a VM. Let m_i represent the used memory configuration for the SF when $p_i = 1$. When $p_i = 0$, let μ_i denote the used VM instance, and θ_i represent the corresponding type.

If source task n_i and destination task n_j of data edge $e_{i,j}$ are deployed on the same VM, data transfer time $d_{i,j}$ is 0; otherwise, it is calculated as $q_{i,j}/w$, where w denotes the transmission bandwidth between cloud resources.

For task n_i deployed on an SF, it can be scheduled for execution once all of its predecessors have completed and the corresponding data transfers are finished. If it is deployed on VM μ_i, it is additionally required that μ_i is available at that time. Let $t_\bullet(n_j)$ denote n_j's finish time, and $\mathcal{A}_i$ represent the earliest available time of VM μ_i to execute n_i. n_i's start time, denoted as $t_\text{o}(n_i)$, is determined as follows:

$$t_\text{o}(n_i) = \begin{cases} \max\left\{\max\limits_{n_j \in *n_i}\left(t_\bullet(n_j) + d_{j,i}\right),\ \mathcal{A}_i\right\}, & \text{if } p_i = 0 \\ \max\limits_{n_j \in *n_i}\left(t_\bullet(n_j) + d_{j,i}\right), & \text{if } p_i = 1 \end{cases}. \tag{3}$$

$t_\bullet(n_i)$ can thus be obtained via,

$$t_\bullet(n_i) = t_\text{o}(n_i) + \begin{cases} r_{i,\theta_i}, & \text{if } p_i = 0 \\ \tilde{r}_{i,m_i} + \tilde{l}_{i,m_i}, & \text{if } p_i = 1 \end{cases}. \tag{4}$$

The total execution time of a workflow, also known as its makespan, is determined as,

$$\mathcal{M} = t_\bullet(n_\text{exit}) - t_\text{o}(n_\text{entry}). \tag{5}$$

The total execution cost of the workflow can be obtained by aggregating the costs of each task deployed on the serverless platform and the costs of all utilized VMs, i.e.,

$$\mathcal{C} = \sum_{n_i \in N,\ p_i = 1} \tilde{c}_{i,m_i} + \sum_{v_k \in V} c_k. \tag{6}$$

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:20:00 UTC from IEEE Xplore. Restrictions apply.



Fig. 4. The flow chart of BWB.

We aim to minimize the workflow makespan subject to the specified budget limit B, that is,

$$\begin{aligned} &\min \quad \mathscr{M} \\ &\text{s.t. } \mathscr{C} \leq B. \end{aligned} \tag{7}$$

The workflow scheduling solution is denoted as $Z = (z_1, z_2, \ldots, z_{|N|})$, where $z_i = \langle n_i, p_i, x, t_o(n_i), t_\bullet(n_i)\rangle$ is a 5-tuple representing the task-resource mapping. Specifically, if $p_i = 0$, it indicates that task n_i is deployed on the x-th VM during the time interval $[t_o(n_i), t_\bullet(n_i)]$. If $p_i = 1$, it means that n_i is executed on an SF with memory configuration x over the same time interval.

IV. Optimization Approach

In this section, we present BWB for solving the aforementioned constrained workflow scheduling problem. As illustrated in Fig. 4, it operates through three consecutive phases: 1) VM-based scheduling generates an initial solution using VMs; 2) SF-based cost reduction lowers expenses by strategically replacing VMs with SFs; and 3) CP-based constrained optimization iteratively refines configurations for critical-path tasks to achieve performance optimization within budget constraints.

Algorithm 1 VM-Based Scheduling

Input: DAG $\langle N, E\rangle$, budget B;
Output: scheduling solution Z;
1: Initialize Z as an array, $\hat{V} \leftarrow \phi$;
2: $b \leftarrow B$, $r_{\text{sum}} \leftarrow \sum_{n_i \in N} \bar{r}_i$;
3: $L \leftarrow$ sort tasks in N in non-increasing order of ρ;
4: **for** each $n_i \in L$ **do**
5: Let $\hat{V}$ be a union of VMs used in Z and V_{new};
6: **for** each $v_k \in \hat{V}$ **do**
7: Calculate $t_\bullet(n_i)$, $\Delta C(n_i, v_k)$;
8: **end for**
9: $\beta_i \leftarrow b \times \bar{r}_i / r_{\text{sum}}$;
10: $v_* \leftarrow$ choose the best VM in $\hat{V}$ based on β_i;
11: Calculate $t_o(n_i)$ and $t_\bullet(n_i)$ on v_*;
12: $z_i \leftarrow \langle n_i, 0, v_*, t_o(n_i), t_\bullet(n_i)\rangle$;
13: $b \leftarrow b - \Delta C(n_i, v_*)$, $r_{\text{sum}} \leftarrow r_{\text{sum}} - \bar{r}_i$;
14: **end for**
15: return Z;

A. Schedule Initialization

To generate an initial schedule, we only use the serverful cloud resources to execute the workflow based on the classic list scheduling heuristic [30]. It is realized in Algorithm 1. The scheduling solution Z is initialized as an array of task-resource mappings of size $|N|$. The set of candidate VMs $\hat{V}$ is initially empty. Let $\bar{r}_i$ be the average computation time of task n_i on all considered types of VMs. The remaining budget b is set to the total budget B, and the cumulative average runtime of all tasks, denoted by r_{sum}, is computed (Line 2). Tasks in the workflow are first sorted in non-increasing order of their upward rank values [30] to generate scheduling list L (Line 3). The upward rank ρ_i of n_i, measuring the length of the longest path from n_i to n_{exit}, can be recursively computed as follows:

$$\rho_i = \max_{n_j \in n_i *} \left[\rho_j + d_{i,j}\right] + \bar{r}_i. \tag{8}$$

Subsequently, the tasks in L are traversed one by one for allocation to VMs. Given that the cloud theoretically provides an unlimited number of VM instances, the candidate VM set ($\hat{V}$) consists of all VMs already utilized in Z and an additional set of newly provisioned instances, V_{new}, where one extra instance is introduced for each VM type (Line 5). For task n_i, all VMs in $\hat{V}$ are evaluated. For VM v_k, the finish time $t_\bullet(n_i)$ and the corresponding cost increase $\Delta C(n_i, v_k)$ incurred by allocating n_i to v_k are calculated (Lines 6-8). An insertion policy is applied when allocating n_i to v_k: if an idle time slot among existing tasks on v_k is sufficiently long to accommodate n_i, it is inserted into that slot; otherwise, it is appended to the end of v_k's schedule. Due to MBT in the VM billing mechanism, $\Delta C(n_i, v_k)$ can be 0 for certain VMs.

After evaluating all VMs in $\hat{V}$, a per-task budget constraint β_i is assigned to n_i, computed as $b \times \bar{r}_i / r_{\text{sum}}$ (Line 9). Based on this, the best VM v_* is selected according to the following criterion (Line 10): if there exist VMs satisfying the per-task budget constraint (i.e., $\Delta C(n_i, v_k) < \beta_i$), the one with the earliest finish time is chosen; among them, if multiple VMs have the same earliest finish time, the one with the lowest cost increase is selected. Otherwise, the VM that incurs the minimal cost increment is chosen.

Algorithm 2 SF-Based Cost Reduction

Input: DAG $\langle N, E\rangle$, scheduling solution Z;
Output: updated scheduling solution Z;
1: **for** each VM v_k used in Z **do**
2: Calculate σ_k by using SFs;
3: **if** $c_k > \sigma_k$ **then**
4: **for** each n_i assigned to v_k in Z **do**
5: $z_i \leftarrow \langle n_i, 1, m_i *$, null, null$\rangle$;
6: **end for**
7: **end if**
8: **end for**
9: **for** each $n_i \in L$ **do**
10: Calculate $t_o(n_i)$ and $t_\bullet(n_i)$ via (3- 4);
11: Update z_i;
12: **end for**
13: return Z;

Afterward, n_i's start and finish time on v_* are calculated via (3) and (4) based on the insertion policy, and the corresponding scheduling information is recorded in Z (Lines 11-12). Meanwhile, b and r_{sum} are updated (Line 13). When all the tasks are scheduled, the initial solution Z is returned.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:20:00 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 CP-Based Constrained Optimization

Input: DAG $\langle N, E\rangle$, budget B, scheduling solution Z;
Output: updated scheduling solution Z;
1: $b \leftarrow B - C(Z)$, $\Phi \leftarrow \emptyset$;
2: **while** $b > 0$ **do**
3: Determine the CP based on Z;
4: $N_{\text{CP}} \leftarrow$ set of tasks in CP;
5: **if** $N_{\text{CP}} \setminus \Phi = \emptyset$ **then**
6: break;
7: **end if**
8: **for** each $n_i \in N_{\text{CP}} \setminus \Phi$ **do**
9: Calculate α_i based on $\mathcal{S}_i$ and $\mathcal{R}_i$;
10: **end for**
11: $n_* \leftarrow \underset{n_i \in N_{\text{CP}} \setminus \Phi}{\text{argmax}}\ \alpha_i$;
12: Calculate β_* for n_*;
13: Determine δ'_* for n_* based on β_*;
14: **if** $\Delta C > b$ **then**
15: $\Phi \leftarrow \Phi \cup \{n_*\}$;
16: continue;
17: **end if**
18: Update Z by assigning n_* to an SF with δ'_*;
19: Update Z by recalculating $t_{\text{o}}(n_i)$ and $t_{\bullet}(n_i)$ for tasks in L from n_* to the end;
20: $b \leftarrow b - \Delta C$;
21: **end while**
22: return Z;

B. SF-Based Cost Reduction

In this phase, schedule Z is revised by replacing VMs with SFs, as described in Algorithm 2. For task n_i, the most cost-efficient serverless configuration is identified by minimizing (2), i.e.,

$$m_i* = \underset{m}{\text{argmin}}\ \tilde{c}_{i,m}. \tag{9}$$

For each VM v_k used in Z, the total cost σ_k of executing all tasks assigned to it by using SFs with their most cost-effective configurations is computed. If the current VM cost c_k is larger than σ_k, all tasks mapped to v_k are migrated to SFs for cost reduction (i.e., setting $p_i = 1$), and v_k is released (Lines 3-7).

Since migrating tasks to SFs may alter their execution durations, the start and finish time of all tasks must be re-calculated by traversing the task list L, and the updated solution Z is returned (Lines 9-13).

C. CP-Based Constrained Optimization

For a given workflow and a specific scheduling solution, makespan is determined by the execution time of the tasks along the critical path (CP). This phase aims to improve workflow performance by adjusting the serverless configurations of tasks on CP, as shown in Algorithm 3. Let $C(Z)$ denote the cost determined by Z. The remaining budget b is initialized as $B - C(Z)$, and the taboo task set Φ, containing tasks excluded from further optimization, is initially set to $\emptyset$ (Line 1).

As long as $b > 0$, the current solution is iteratively refined by using the remaining budget. In each iteration, CP of the workflow graph is recalculated based on Z, and N_{CP} denotes the set of tasks on CP (Lines 3-4). If $N_{\text{CP}} \setminus \Phi = \emptyset$, the loop terminates (Lines 5-7).

For each task in $N_{\text{CP}} \setminus \Phi$, the refinement potential α_i is assessed from the graph structure sensitivity $\mathcal{S}_i$ and the refinement sensitivity $\mathcal{R}_i$ (Lines 8-10). Specifically, $\mathcal{S}_i$ represents the upper bound on the improvement in the overall workflow makespan that can be achieved by refining n_i. It is calculated as the makespan of Z minus the makespan obtained by setting n_i's runtime to zero. $\mathcal{R}_i$ represents the reduction in n_i's runtime per unit increase in memory size when $p_i = 1$. It is calculated as the negative slope of the memory–runtime curve at point m_i. If m_i is already at the maximum allowed memory configuration, then $\mathcal{R}_i = 0$. When $p_i = 0$, two adjacent memory configurations m and $m+1$ on the memory–runtime curve are identified such that $\tilde{r}_{i,m} \geq r_{i,\theta_i} > \tilde{r}_{i,m+1}$, and the value of $\mathcal{R}_i$ at point m is then used. The refinement potential α_i is calculated as $\mathcal{S}_i \times \mathcal{R}_i$.

Then, the task with the largest refinement potential, denoted as n_*, is chosen for refinement (Line 11) and its refinement potential is denoted by α_*. The remaining budget is proportionally distributed to n_* based on its refinement potential (Line 12), i.e.,

$$\beta_* = b \times \frac{\alpha_*}{\sum_{n_i \in N_{\text{CP}} \setminus \Phi} \alpha_i}. \tag{10}$$

The updated configuration δ'_* of n_* is selected as the one that yields the shortest runtime while satisfying the cost increment constraint β_* (Line 13). If no such configuration exists due to an insufficient β_*, n_* is reassigned to the larger configuration that offers the lowest cost.

If the resulting cost increment ΔC exceeds the remaining budget b, n_* is added to the taboo task set Φ, and the current iteration is skipped (Lines 14–17). Otherwise, the solution Z is updated by assigning n_* to an SF with configuration δ'_* (Line 18). Subsequently, $t_{\text{o}}(n_i)$ and $t_{\bullet}(n_i)$ for tasks in L from n_* to the end are recalculated via (3) and (4) (Line 19). The remaining budget b is updated accordingly (Line 20). Once the loop terminates, the refined scheduling solution is returned (Line 22).

D. Complexity Analysis

Suppose that the workflow has $|N|$ tasks, $|E|$ directed edges, each task can be configured with $|M|$ kinds of memories, and $|\mathcal{T}|$ types of VMs are available. In Algorithm 1, the computation of upward ranks for all tasks has a time complexity of $\mathcal{O}(|N| + |E|)$, and the sorting operation has a time complexity of $\mathcal{O}(|N| \cdot \log |N|)$. When deploying task n_i, each time slot and the end of each VM should be considered, and hence the time complexity for computing $\Delta C(n_i, v_k)$, and selecting the optimal VM is $\mathcal{O}(|N| + |\mathcal{T}|)$. The time complexity of this phase is: $\mathcal{O}(|N| + |E| + |N| \cdot \log |N| + |N|^2 + |N| \cdot |\mathcal{T}|) = \mathcal{O}(|N|^2 + |N| \cdot |\mathcal{T}|)$ as $|E| \leq |N|^2$.

In Algorithm 2, the time complexity of computing each σ_k is $\mathcal{O}(|N|)$. At most, $|N|$ tasks may be migrated to SFs, and updating the scheduling solution after replacement has a time complexity of $\mathcal{O}(|N| + |E|)$. Therefore, the time complexity of Algorithm 2 is $\mathcal{O}(|N| + |E|)$.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:20:00 UTC from IEEE Xplore. Restrictions apply.

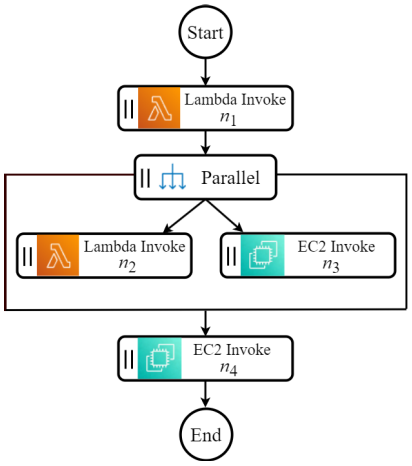


Fig. 5. The state machine of the workflow with blended cloud resources.

In Algorithm 3, the time complexity for identifying CP in each iteration is $\mathcal{O}(|N| + |E|)$. Since the number of candidate tasks is at most n, and the number of paths is also bounded by n, the complexity of calculating the refinement potential is $\mathcal{O}((|N| + |E| + |N|^2) \cdot h) = \mathcal{O}(|N|^2 \cdot h)$, assuming a maximum of h iterations. For tuning configurations, the worst-case scenario requires computing the updated workflow makespan and cost exactly once for each configuration of each task, resulting in an overall time complexity of $\mathcal{O}(|E| \cdot |N| \cdot |M|)$. Thus, Algorithm 3 has a time complexity of $\mathcal{O}(|E| \cdot |N| \cdot |M| + |N|^2 \cdot h)$.

Combining all the above results, we have the time complexity of BWB: $\mathcal{O}(|E| \cdot |N| \cdot |M| + |N|^2 \cdot h + |N| \cdot |\mathcal{T}|)$.

V. Experimental Evaluations

This section presents both real-world and simulation experiments conducted to validate the proposed approach and discusses several key findings.

A. Real-World Experiments

Verification experiments are conducted to execute the scheduling solution yielded by BWB in a realistic blended cloud environment comprising AWS EC2 and AWS Lambda. Specifically, according to timing rules and memory configuration in a solution, VM resources are automatically scheduled to start and stop, and the memory settings of SFs are dynamically configured via the AWS Command Line Interface. Then, the workflow coordinator tool AWS Step Functions[3] is used to execute the serverless workflow as a state machine, in which each task is configured to run on either a VM or SF. For example, the scheduling solution in Fig. 1 (d) can be converted into the state machine depicted in Fig. 5. To avoid potential cross–Availability Zone data transfer charges, all VMs and SFs are deployed within the same Availability Zone of a single AWS region.

We select four realistic workflow applications from the Pegasus workflow gallery—Epigenomics, SoyKB, Inspiral, and CyberShake—which are used for studying epigenetic features of genomes, soybean-related bioinformatics, biological data, and seismic hazards, respectively [31]. They contain 45, 44, 50, and 1000 tasks, respectively, and exhibit significant differences in structure, communication patterns, and computational characteristics. Their graph structures are illustrated in Fig. 6. For tasks in these workflows, their runtime on a standard computation service is provided in the gallery. To ensure consistent execution durations, we simulate task execution by implementing sleep operations of equivalent lengths in both SF and VM environments based on the recorded runtime. In the SF environment, the sleep durations are proportionally reduced by scaling up the memory configuration.

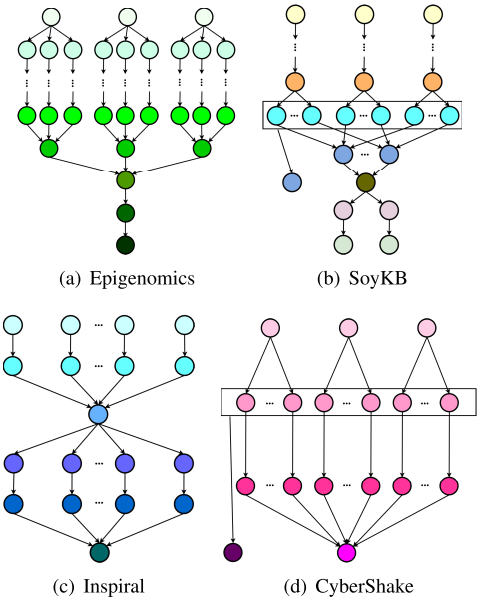


Fig. 6. DAG structures of the Epigenomics, SoyKB, Inspiral and CyberShake workflows.

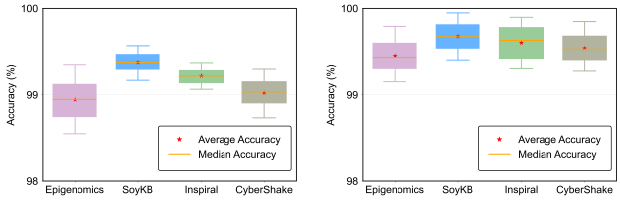


Fig. 7. Accuracy results.

Based on the above setup, a scheduling solution is generated by BWB for each workflow and executed with cold starts in the realistic blended cloud environment 100 times. The actual makespan and cost values are recorded. For comparison, the makespan and cost are also estimated by using the proposed scheduling model.

The actual values are compared with the estimated values, and the accuracy is visualized by using box plots, as shown in Fig. 7. It can be observed that the accuracy values are very high: the average makespan accuracy exceeds 98.5%, and the average cost accuracy exceeds 99.0%. The results indicate that the overhead of cloud workflow orchestration is negligible compared to the workflow makespan. The inconsistency mainly arises from the variability in SF cold start latency and VM launch latency. The accuracy is the lowest

[3]https://aws.amazon.com/step-functions

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:20:00 UTC from IEEE Xplore. Restrictions apply.

for Epigenomics because its makespan is the shortest, making the impact of latency variance more pronounced.

B. Simulation Settings

To conduct repeatable and statistically significant comparison experiments between the proposed method and existing approaches across a wide range of configurations, a blended cloud environment is simulated. Specifically, the simulation experiments are carried out on a Windows 11 PC equipped with an i7-13700HX 2.1GHz CPU and 16GB RAM.

For the serverful platform in the simulated blended cloud environment, MBT and VM startup latency (l_{VM}) are set to 60 seconds and 30 seconds, respectively, to align with real-world settings in Amazon EC2. Similar to [12], six VM types located within the USA East region (N. Virginia) on Amazon EC2 are used, and their processing capacities and unit prices per second ($\$/s$) are listed in Table II. As the tasks in the above four workflows are characterized by runtime on a standard computation service instead of task workloads, we represent the capacity of each VM type as a multiple of the speed of this standard service. Accordingly, the runtime of a task on a specific VM type equals the standard runtime divided by the corresponding capacity.

For the serverless platform, the price per GB-second (PGS) and the per-invocation cost (PPI) are set to 0.0000166667 USD/(GB·s) and 0.0000002 USD, respectively. To capture the variability of startup delays across different serverless functions, the cold start latency is modeled as an exponential distribution with a mean of 1.5 seconds. The bandwidth (w) between two cloud resources is set to 10 Gbps. For the four workflows used, the task runtime on SFs with 128MB memory is set to the corresponding value available in the Pegasus gallery, and gradually reduced as the allocated memory size increases (up to 10,240MB), following the scaling trends described in [19] and [22].

BWB is compared against two existing approaches designed for blended cloud environments, namely BCWS [9] and Mashup [8]. In addition, two benchmark methods are included for comparison: UWC [20], which utilizes only serverless resources, and PACP-HEFT [12], which relies solely on serverful resources. The implementations strictly follow the algorithm descriptions in their respective studies, with hyperparameters set as recommended in [8], [9], [12], and [20].

To evaluate the scheduling algorithm under varying budget conditions, we introduce a budget factor $\varepsilon \in [0, 1]$ to represent the degree of budget flexibility. The workflow budget is then determined based on ε, as defined by

$$B = C_{\min} + \varepsilon \times (C_{\max} - C_{\min}), \tag{11}$$

where $C_{\min}$ and $C_{\max}$ represent the monetary cost of executing the workflow when each task is run on an SF with the lowest and highest memory configuration, respectively.

C. Experimental Results and Analysis

We first conduct experiments by varying the budget factor ε, and then perform sensitivity analysis under different bandwidth settings w, as well as robustness analysis under profiling inaccuracies. Owing to the stochastic nature of the experimental setup, each experiment is repeated 20 times, and the mean performance with standard deviations are reported.

TABLE II
THE PROCESSING CAPACITY AND CHARGE FOR VM TYPES USED IN EXPERIMENTS

No.	Type name	Processing capacity	Unit price($\$/s$)
1	m1.small	1	1.22E-5
2	m1.medium	2	2.42E-5
3	m3.medium	3	1.86E-5
4	m1.large	4	4.86E-5
5	m3.large	6.5	3.69E-5
6	m1.xlarge	8	9.72E-5

TABLE III
AVERAGE WORKFLOW MAKESPAN (S) FROM EACH SCHEDULING METHOD

Workflow	BWB	BCWS	UWC	Mashup	PACP-HEFT
Epigenomics	79.10	99.10	99.96	139.75	114.29
SoyKB	1039.02	1077.56	1451.41	1832.02	1365.53
Inspiral	699.76	776.13	788.03	871.01	923.88
CyberShake	348.39	382.08	378.52	622.39	561.37

TABLE IV
SIGNIFICANCE TEST OF BWB OVER COMPARED ALGORITHMS

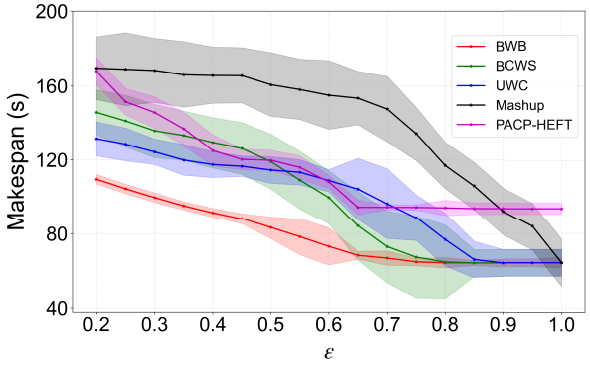
Workflow	BCWS	UWC	Mashup	PACP-HEFT
Epigenomics	-	-	-	-
SoyKB	=	-	-	-
Inspiral	-	-	-	-
CyberShake	-	-	-	-

1) Impact of Varying Budget Factor ε: To compare the performance of the five scheduling methods, we vary ε from 0.2 to 1 in increments of 0.05. Fig. 8 presents their mean makespan results with standard deviations across the four workflows. As ε increases, a larger budget becomes available, leading to shorter makespans for all approaches. BWB consistently outperforms its peers and its standard deviations are relatively small. For instance, when $\varepsilon = 0.3$, BWB achieves a mean makespan of less than 400s for the CyberShake workflow, whereas all other methods result in makespans exceeding 480 seconds.

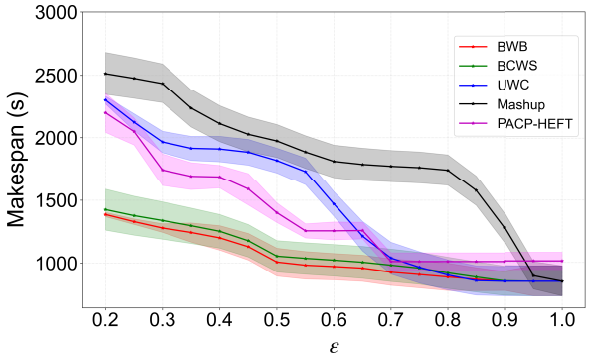
We further report the average of the mean makespan values for each scheduling method across all ε values in Table III. The results show that BWB achieves average makespan reductions of 10.6%, 17.1%, 37.6%, and 29.2% over BCWS, UWC, Mashup, and PACP-HEFT, respectively. Besides, we conduct statistical significance tests between BWB and the compared methods via the Wilcoxon signed-rank test at a significance level of 0.05 [32]. The results are presented in Table IV, where "-", "=", and "+" means that BWB is significantly better, similar, and significantly worse than the compared method in terms of makespan, respectively. The results demonstrate that BWB significantly outperforms its peers.

PACP-HEFT and UWC generally perform poorly, as they rely exclusively on either serverful or serverless cloud resources. This highlights the effectiveness of leveraging

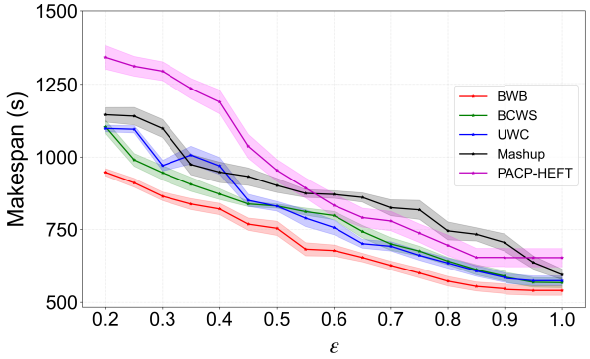
Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:20:00 UTC from IEEE Xplore. Restrictions apply.



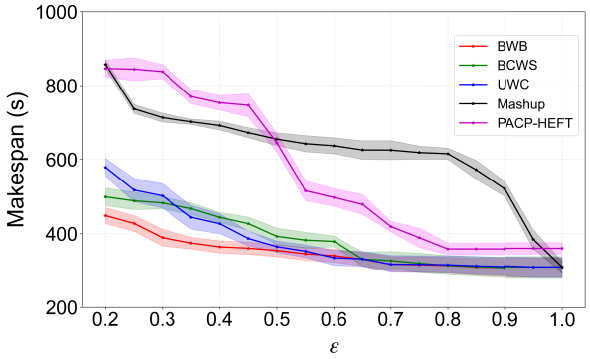
(a) Epigenomics



(b) SoyKB



(c) Inspiral



(d) CyberShake

Fig. 8. Makespan results with regard to budget factor ε.

blended cloud resources for accelerated workflow execution. Mashup performs worse than the other two blended cloud methods, as it primarily focuses on selecting the preferred execution platform for individual tasks within a workflow. In contrast, BCWS iteratively refines a solution by updating task configurations. However, it naively treats the execution time and cost variations of individual tasks as those of the overall workflow, which results in its suboptimal tuning decisions. BWB delivers the best overall performance by effectively leveraging the complementary strengths of serverful and serverless cloud services.

TABLE V
RUNTIME OVERHEAD (MS) OF EACH SCHEDULING METHOD

Workflow	BWB	BCWS	UWC	Mashup	PACP-HEFT
Epigenomics	5.7	8.5	3.4	5.1	2.8
SoyKB	7.2	8.8	4.8	4.4	3.1
Inspiral	4.6	6.6	4.3	4.8	2.5
CyberShake	1571.9	1816.7	853.2	743.8	697.8

In addition, the runtime overhead required by each method to generate scheduling solutions is recorded, and the mean results are summarized in Table V. For the medium-sized workflows Epigenomics, SoyKB and Inspiral, the runtime of each method is less than 10 ms. For the large-scale workflow CyberShake with 1,000 tasks, each method takes approximately 1 second to generate a solution. The runtime of BWB is basically similar to those of the compared methods, while achieving superior scheduling performance. Among all the methods, PACP-HEFT requires the shortest runtime, as it relies solely on the VM platform. UWC and Mashup exhibit relatively short runtime since their optimization does not involve task switching across platforms. In contrast, BCWS incurs the longest runtime due to task execution platform switching during optimization.

2) Sensitivity Analysis Under Varying Bandwidth w: Different network bandwidth settings can significantly affect workflow performance. Therefore, we set the bandwidth w to 0.1 Gbps, 1 Gbps, and 10 Gbps, respectively, and conduct additional comparative experiments. All methods are re-evaluated under the same settings with $\varepsilon = 0.5$, and the results are reported in Table VI. As the bandwidth decreases, the makespan of the workflow schedules generated by all methods increases. For example, for the CyberShake workflow, which involves large volumes of data transfer, the execution time under BWB increases substantially, from 354.80 when $w = 10$ Gbps to 3210.80 when $w = 0.1$ Gbps. Regardless of the bandwidth setting, our method BWB consistently outperforms its peers for all workflows.

3) Robustness Analysis Under Profiling Inaccuracies: Since task runtime and memory–performance profiling data may not be completely accurate in practical cloud environments, the robustness of scheduling methods is evaluated under imperfect profiling conditions. Specifically, different levels of profiling inaccuracies are simulated by injecting bounded random errors (i.e.,$\pm$5%, $\pm$10%, $\pm$20%) into the profiled values. All methods are re-evaluated with $\varepsilon = 0.5, w = 10$, with the results listed in Table VII. Compared with the corresponding results in Table VI obtained by using perfect profiling data, the makespan of each method increases in Table VII due to

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:20:00 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
WORKFLOW MAKESPAN (S) UNDER DIFFERENT BANDWIDTHS

Workflow	w	BWB	BCWS	UWC	Mashup	PACP-HEFT
Epigenomics	0.1	321.97±10.48	337.52±18.64	358.68±6.46	369.33±15.30	358.11±5.17
	1	112.08±6.44	137.04±12.66	139.65±5.09	168.43±17.33	148.67±7.34
	10	83.30±5.97	118.99±14.75	114.51±7.19	160.66±17.17	119.78±5.46
SoyKB	0.1	2542.32±123.02	2866.51±134.55	3021.25±113.65	3623.33±152.45	3158.69±85.91
	1	1355.67±104.36	1588.63±134.36	2011.39±108.66	2341.97±135.77	1952.36±81.39
	10	1007.93±109.91	1054.64±123.84	1818.14±98.42	1973.80±130.66	1400.17±77.95
Inspiral	0.1	1845.22±35.21	2088.65±13.48	2155.64±23.59	2254.89±27.51	2369.22±41.39
	1	955.30±28.19	1074.35±9.47	1142.08±21.84	1285.34±36.08	1365.32±39.14
	10	753.85±24.66	831.02±7.34	830.22±16.79	902.03±23.95	953.66±37.37
CyberShake	0.1	3210.80±16.01	3396.41±26.11	3349.66±17.24	3756.29±19.38	3794.10±35.28
	1	652.34±17.63	702.33±23.98	814.30±18.37	1013.09±24.31	984.67±29.64
	10	354.80±16.60	392.92±22.33	365.22±15.35	655.10±17.35	645.06±25.09

TABLE VII
WORKFLOW MAKESPAN (S) UNDER DIFFERENT PROFILING INACCURACIES

Workflow	Inaccuracy	BWB	BCWS	UWC	Mashup	PACP-HEFT
Epigenomics	5%	87.51±8.94	124.69±17.99	113.55±5.84	169.53±19.52	124.92±6.59
	10%	85.15±12.07	116.71±26.39	117.66±7.13	164.32±24.07	119.64±13.66
	20%	91.20±17.44	127.88±31.21	123.54±14.99	176.38±28.11	128.35±18.17
SoyKB	5%	1054.61±117.30	1185.66±141.02	1795.20±112.05	2042.99±134.81	1469.99±68.35
	10%	1087.38±131.50	1192.07±132.64	1830.94±152.32	1985.34±148.37	1415.08±91.54
	20%	1089.65±162.84	1123.41±169.30	1886.67±167.39	2166.67±167.08	1384.39±108.22
Inspiral	5%	784.25±27.32	854.36±13.95	861.33±18.41	933.89±36.12	967.55±34.62
	10%	801.36±35.66	884.89±15.84	865.35±25.66	918.34±31.84	1012.44±47.28
	20%	795.01±41.09	851.44±11.95	874.08±39.69	925.10±47.66	984.08±89.31
CyberShake	5%	362.30±19.41	397.95±25.39	371.91±13.77	688.20±23.55	708.05±28.45
	10%	361.09±23.37	411.68±26.30	369.38±19.94	673.90±19.36	677.91±34.60
	20%	379.94±31.54	421.08±35.98	389.91±29.33	702.05±36.24	684.33±38.11

inaccurate profiling data. As the level of profiling inaccuracy increases, the standard deviation of the resulting makespan generally increases across all methods. Nevertheless, BWB performs the best among all compared methods in all cases.

VI. CONCLUSION AND FUTURE WORK

This paper presents a novel workflow scheduling framework for a blended cloud environment that integrates complementary serverful and serverless resources for cost-efficient workflow execution. The proposed algorithm called BWB aims to minimize the makespan under a user-defined budget constraint. It begins by constructing an initial solution using VM resources, then strategically reduces cost by replacing selected VMs with SFs, and finally iteratively refines the configurations of critical-path tasks to further improve performance. Extensive experiments conducted with real-world cloud settings and realistic workflow applications demonstrate the effectiveness of blending cloud resources for workflow execution and confirm the superiority of BWB over the state-of-the-art approaches.

For future work, we plan to extend our blended cloud scheduling solution to support a broader range of application scenarios such as Internet of Things [33], machine learning [34], deep learning [35] and large language models [36], [37]. In addition, we aim to investigate failure handling mechanisms and establish theoretical guarantees for the scheduling algorithms.

ACKNOWLEDGMENT

The authors would like to thank the editor and anonymous reviewers for their valuable comments/suggestions that help them to improve this article.

REFERENCES

[1] H. Shafiei, A. Khonsari, and P. Mousavi, "Serverless computing: A survey of opportunities, challenges and applications," *ACM Comput. Surv.*, pp. 1–32, 2020.
[2] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, "Cold start latency in serverless computing: A systematic review, taxonomy, and future directions," *ACM Comput. Surv.*, vol. 57, no. 3, pp. 1–36, Mar. 2025.
[3] M. Armbrust et al., "A view of cloud computing," *Commun. ACM*, vol. 53, no. 4, pp. 50–58, 2010.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:20:00 UTC from IEEE Xplore. Restrictions apply.

[4] H. Li, D. Wang, M. Zhou, Y. Fan, and Y. Xia, "Multi-swarm co-evolution based hybrid intelligent optimization for bi-objective multi-workflow scheduling in the cloud," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 9, pp. 2183–2197, Sep. 2022.

[5] K. Cai, Q. Wu, M. Zhou, C. Chen, J. Wen, and S. Wang, "Dynamically scheduling deadline-constrained interleaved workflows on heterogeneous computing systems," *IEEE Trans. Services Comput.*, vol. 18, no. 2, pp. 758–769, Mar. 2025.

[6] S. Kulagina, H. Meyerhenke, and A. Benoit, "Mapping large memory-constrained workflows onto heterogeneous platforms," in *Proc. 53rd Int. Conf. Parallel Process.*, Aug. 2024, pp. 305–316.

[7] P. Verma, A. K. Maurya, and R. S. Yadav, "A survey on energy-efficient workflow scheduling algorithms in cloud computing," *Softw., Pract. Exper.*, vol. 54, no. 5, pp. 637–682, 2023.

[8] R. B. Roy, T. Patel, V. Gadepally, and D. Tiwari, "Mashup: Making serverless computing useful for HPC workflows via hybrid execution," in *Proc. 27th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, Apr. 2022, pp. 46–60.

[9] J. Zhang, X. Li, L. Chen, and R. Ruiz, "Scheduling workflows with limited budget to cloud server and serverless resources," *IEEE Trans. Services Comput.*, vol. 17, no. 4, pp. 1766–1779, Jul. 2024.

[10] M. Tasleem Khan and O. Gustafsson, "Stochastic analysis of LMS algorithm with delayed block coefficient adaptation," 2023, *arXiv:2306.00147*.

[11] H. R. Faragardi, M. R. Saleh Sedghpour, S. Fazliahmadi, T. Fahringer, and N. Rasouli, "GRP-HEFT: A budget-constrained resource provisioning scheme for workflow scheduling in IaaS clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 6, pp. 1239–1254, Jun. 2020.

[12] M. Fan, X. Zhao, X. Zuo, and L. Ye, "A budget-constrained workflow scheduling approach with priority adjustment and critical task optimizing in clouds," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 6907–6921, 2025.

[13] Z. Sun, Y. Mei, F. Zhang, H. Huang, C. Gu, and M. Zhang, "Multi-tree genetic programming hyper-heuristic for dynamic flexible workflow scheduling in multi-clouds," *IEEE Trans. Services Comput.*, vol. 17, no. 5, pp. 2687–2703, Sep. 2024.

[14] X. Shu, Q. Wu, M. Zhou, and J. Wen, "A communication-contention-aware privacy-preserving workflow scheduling method for geo-distributed datacenters," *IEEE Trans. Services Comput.*, vol. 17, no. 5, pp. 1887–1898, Sep. 2024.

[15] C. Lu, J. Zhu, H. Huang, and Y. Sun, "A multi-hierarchy particle swarm optimization-based algorithm for cloud workflow scheduling," *Future Gener. Comput. Syst.*, vol. 153, pp. 125–138, Apr. 2024.

[16] N. Rizvi, D. Ramesh, L. Wang, and A. Basava, "A workflow scheduling approach with modified fuzzy adaptive genetic algorithm in IaaS clouds," *IEEE Trans. Services Comput.*, vol. 16, no. 2, pp. 872–885, Mar. 2023.

[17] S. Qin, D. Pi, Z. Shao, Y. Xu, and Y. Chen, "Reliability-aware multi-objective memetic algorithm for workflow scheduling problem in multi-cloud system," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 4, pp. 1343–1361, Apr. 2023.

[18] Q. Wu, M. Zhou, Q. Zhu, Y. Xia, and J. Wen, "MOELS: Multiobjective evolutionary list scheduling for cloud workflows," *IEEE Trans. Autom. Sci. Eng.*, vol. 17, no. 1, pp. 166–176, Jan. 2020.

[19] C. Lin and H. Khazaei, "Modeling and optimization of performance and cost of serverless applications," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 3, pp. 615–632, Mar. 2021.

[20] Z. Li, H. Yu, and G. Fan, "Time-cost efficient memory configuration for serverless workflow applications," *Concurrency Comput., Pract. Exper.*, vol. 34, no. 27, p. 7308, Dec. 2022.

[21] Y. Jia, C. Jin, Q. Li, X. Liu, and X. Jin, "FaaSPR: Latency-oriented placement and routing optimization for serverless workflow processing," *IEEE Trans. Netw.*, vol. 33, no. 4, pp. 2063–2078, Aug. 2025.

[22] W. Wang, Q. Wu, Z. Zhang, J. Zeng, X. Zhang, and M. Zhou, "A probabilistic modeling and evolutionary optimization approach for serverless workflow configuration," *Softw., Pract. Exper.*, vol. 54, no. 9, pp. 1697–1713, Sep. 2024.

[23] M. Bilal, M. Canini, R. Fonseca, and R. Rodrigues, "With great freedom comes great opportunity: Rethinking resource allocation for serverless functions," in *Proc. 18th Eur. Conf. Comput. Syst.*, May 2023, pp. 381–397.

[24] Z. Zhou, Y. Zhang, and C. Delimitrou, "AQUATOPE: QoS-and-uncertainty-aware resource management for multi-stage serverless workflows," in *Proc. 28th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Dec. 2022, pp. 1–14.

[25] L. Jin, Z. Cai, Z. Chen, H. Zhao, and R. Ma, "AARC: Automated affinity-aware resource configuration for serverless workflows," in *Proc. 62nd ACM/IEEE Des. Automat. Conf. (DAC)*, San Francisco, CA, USA, 2025, pp. 1–7.

[26] A. Reuter, T. Back, and V. Andrikopoulos, "Cost efficiency under mixed serverless and serverful deployments," in *Proc. 46th Euromicro Conf. Softw. Eng. Adv. Appl. (SEAA)*, Aug. 2020, pp. 242–245.

[27] A. Raza, Z. Zhang, N. Akhtar, V. Isahagian, and I. Matta, "LIBRA: An economical hybrid approach for cloud applications with strict SLAs," in *Proc. IEEE Int. Conf. Cloud Eng. (IC2E)*, Oct. 2021, pp. 136–146.

[28] A. Peri, M. Tsenos, and V. Kalogeraki, "Orchestrating the execution of serverless functions in hybrid clouds," in *Proc. IEEE Int. Conf. Autonomic Comput. Self-Organizing Syst. (ACSOS)*, Sep. 2023, pp. 139–144.

[29] A. Mampage, S. Karunasekera, and R. Buyya, "Deep reinforcement learning for scheduling applications in serverless and serverful hybrid computing environments," *IEEE Trans. Services Comput.*, vol. 18, no. 2, pp. 718–728, Mar. 2025.

[30] H. R. Topcuoglu, S. Hariri, and M. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[31] T. Coleman, H. Casanova, L. Pottier, M. Kaushik, E. Deelman, and R. Ferreira da Silva, "WfCommons: A framework for enabling scientific workflow research and development," *Future Gener. Comput. Syst.*, vol. 128, pp. 16–27, Mar. 2022.

[32] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, Dec. 1945.

[33] B. Hu et al., "Hybrid programming-based scheduling approach for many heterogeneous computing tasks with asynchronous generation in IIoT," *IEEE Internet Things J.*, vol. 12, no. 15, pp. 29384–29396, Aug. 2025.

[34] X. Hui, Y. Xu, and X. Shen, "Exploring function granularity for serverless machine learning application with gpu sharing," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 9, no. 1, pp. 1–28, 2025.

[35] M. A. Alhartomi et al., "Low-area and low-power VLSI architectures for long short-term memory networks," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 4, pp. 1000–1014, Dec. 2023.

[36] S. Zeng, M. Xie, S. Gao, Y. Chen, and Y. Lu, "Medusa: Accelerating serverless LLM inference with materialization," in *Proc. 30th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Mar. 2025, pp. 653–668.

[37] J. Bi et al., "Large AI models and their applications: Classification, limitations, and potential solutions," *Softw., Pract. Exper.*, vol. 55, no. 6, pp. 1003–1017, Jun. 2025.



Quanwang Wu (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in computer science from Chongqing University, Chongqing, China, in 2007, 2010, and 2013, respectively. He was a special Researcher with the Digital Content and Media Sciences Research Division, National Institute of Informatics (NII), Tokyo, Japan, from 2014 to 2015. He is currently a Full Professor of the College of Computer Science, Chongqing University. His main research interests include scheduling techniques, services computing, and data mining.



Qixin Zhou received the bachelor's degree in computer science and technology from Chongqing University, Chongqing, China, in 2024, where he is currently pursuing the master's degree in computer science and technology. His main research interests include cloud computing and workflow scheduling.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:20:00 UTC from IEEE Xplore. Restrictions apply.



Ruyi Sun received the bachelor's degree in software engineering from Xinjiang University, Ürümqi, China, in 2024. He is currently pursuing the master's degree in computer science and technology with Chongqing University, Chongqing, China. His main research interests include Bayesian optimization and serverless computing.



Mengchu Zhou (Fellow, IEEE) received the B.S. degree in control engineering from Nanjing University of Science and Technology, Nanjing, China, in 1983, the M.S. degree in automatic control from Beijing Institute of Technology, Beijing, China, in 1986, and the Ph. D. degree in computer and systems engineering from Rensselaer Polytechnic Institute, Troy, NY, USA, in 1990. He joined the Department of Electrical and Computer Engineering, New Jersey Institute of Technology in 1990, and is now a Distinguished Professor. He has over 1400 publications including 20 books, over 900 journal papers including over 700 IEEE Transactions papers, 33 patents and 34 book-chapters. His interests are in intelligent automation/transportation, robotics, Petri nets, Internet of Things, edge/cloud computing, and big data analytics. His recently co-authored books include Scheduling Tasks in Distributed Cloud and Edge Computing Systems with Evolutionary Optimizers (with H. Yuan, J. Bi, and R. Buyya), Springer, Berlin, Germany, 2026; Intelligent Scheduling of Tasks for Cloud-Edge-Device Computing Systems, Wiley-IEEE Press, Hoboken, NJ, 2025 (with B. Hu, Z. Cao, and M. Zhao), and Learning Automata and their Applications to Intelligent Systems, Wiley-IEEE Press, Hoboken, NJ, 2024 (with J. Zhang).

Dr. Zhou is a recipient of Franklin V. Taylor Memorial Award, Norbert Wiener Award and Lotfi A. Zadeh Pioneer Award from IEEE Systems, Man, and Cybernetics Society, Excellence in Research Prize and Medal from NJIT, Humboldt Research Award for US Senior Scientists from Alexander von Humboldt Foundation, and Edison Patent Award from the Research and Development Council of New Jersey. He served as VP for Conferences and Meetings of from IEEE Systems, Man, and Cybernetics Society in 2018-2019. He is a life member of Chinese Association for Science and Technology-USA and served as its President in 1999. He is Fellow of International Federation of Automatic Control (IFAC), American Association for the Advancement of Science (AAAS), Chinese Association of Automation (CAA) and National Academy of Inventors (NAI).



Ji Feng received the B.Sc. degree from Nanjing University of Posts and Telecommunications in 2008 and the Ph.D. degree from Chongqing University in 2016. He is currently an Associate Professor with the College of Computer Science, Chongqing Normal University, Chongqing. His research interests include data mining, artificial intelligence, and smart education. He has published more than 20 refereed journals and conference papers in the above areas.



Taolin Guo received the B.Sc. degree from Sichuan Agricultural University in 2011 and the Ph.D. degree from Southeast University in 2020. He is currently a Lecturer with the College of Computer Science, Chongqing Normal University, Chongqing. His research interests include service computing and privacy protection. He has published more than 20 refereed journals and conference papers in the above areas.



Chao Chen (Senior Member, IEEE) received the B.Sc. and M.Sc. degrees in control science and control engineering from Northwestern Polytechnical University, Xi'an, China, in 2007 and 2010, respectively, and the joint Ph.D. degree from Sorbonne University and Institut Mines-Télécom/Télécom SudParis, France, in 2014. He is currently a Full Professor with the College of Computer Science, Chongqing University, Chongqing, China. His research interests include pervasive computing, mobile computing, data mining from large-scale GPS trajectory data, and big data analytics for smart cities.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:20:00 UTC from IEEE Xplore. Restrictions apply.